



AI with Rav

Learn AI the way you actually think.



DAY 6 of 30

OPEN SOURCE CONTRIBUTION

I Opened 35 Docstring PRs. Two Merged.

WHAT YOU'LL LEARN TODAY

- › Why most documentation PRs are ignored
- › The one question that predicts a merge
- › How to document a function properly
- › Why volume is the wrong strategy



I Opened 35 Docstring PRs. Two Merged.

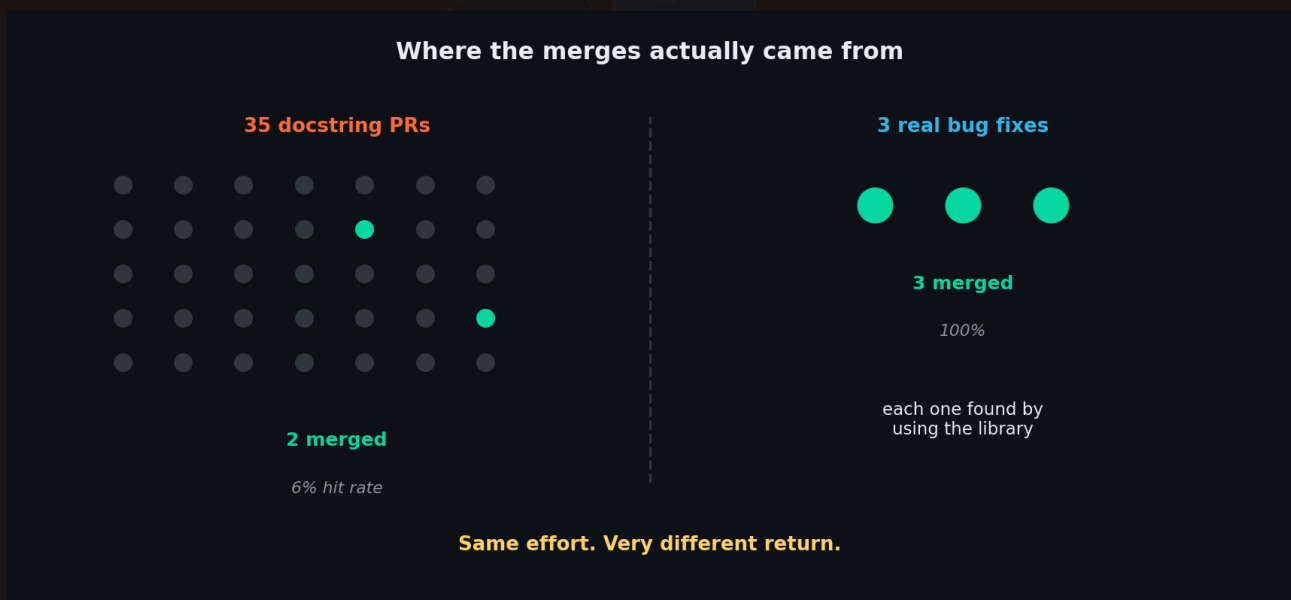
In One Line: Adding docstrings is the most common first contribution and the most commonly rejected one. This is the difference between the ones that merge and the thirty-three that did not.

The uncomfortable number

Before any of the fixes in this series, I opened roughly **thirty-five** documentation PRs across many repositories.

Two merged. This one, and the one in the next video.

I am telling you the number because the advice you usually get — "start with docs, it's the easy way in" — is not wrong exactly, but it leaves out the part that determines whether it works.



Thirty-five PRs, two merges. The same afternoon spent on one real bug had a far better return.

Why most of them failed

Looking back at the ones that died, they fell into three groups:

Pattern	Why it was ignored
Fixing typos in a README	Trivially reversible, no reviewer gain
Rewording docs that were already fine	Now the maintainer must compare two acceptable versions



Adding docs to a repo I had never used

My guess at what a function does is not documentation

That third one is the real killer. If you have not used the code, you are describing what you *think* it does from reading it. A maintainer can tell, and they now have to fact-check your guess — which is slower than writing it themselves.

An unhelpful docs PR is not neutral. It costs the maintainer review time and returns nothing. That is why it gets left open rather than merged.

The question that predicts a merge

Before opening a documentation PR, ask:

Is there a public function here with NO docstring at all, in a module where everything else IS documented?

If yes, you have found a genuine gap and an obvious standard to match. If no — if the docs merely could be *better* — close the tab.

The difference is between **missing** and **improvable**. Missing is a fact. Improvable is an opinion, and opinions need a debate that nobody has time for.

What this PR was

joblib/logger.py. Three public functions with no docstrings, in a module where the rest was documented:

- **format_time** — formats a duration as **'12.3s, 0.2min'**
- **short_format_time** — compact form, switches to minutes above 60s
- **pformat** — pretty-prints an object with numpy options temporarily reduced

Nothing to argue about. They had none, everything around them had some, and the house style was sitting right there in **Logger.__init__** in the same file.

Matching the existing style

I did not choose a docstring format. I copied the one already in the file — NumPy style, Parameters and Returns:



```
def short_format_time(t):
    """Format a duration compactly.

    Parameters
    -----
    t : float
        Duration in seconds.

    Returns
    -----
    str
        The duration formatted as seconds below 60s
        (e.g. '12.3s') and as minutes above (e.g. '1.2min').
    """
```

Never introduce a new documentation style in someone's codebase. If the module uses NumPy style, use NumPy style — even if you prefer Google style. Consistency is the point of a house style.

Getting the content right

The value is in the specifics. Compare:

Weak	Useful
"Formats the time."	"Seconds below 60s ('12.3s'), minutes above ('1.2min')"
"Pretty-prints the object."	"...with numpy print options temporarily reduced"

The left column restates the function name. The right column tells you the **threshold**, the **format**, and the **side effect** — things you would otherwise have to read the source to learn.

If your docstring is just the function name as a sentence, it adds nothing.

"No logic changes"

The PR body ended with three words: **No logic changes.**

Say this explicitly when it is true. It tells the reviewer the risk is zero and they can merge on a quick read rather than reasoning about behaviour.

It also has to be *actually* true. Do not sneak a small fix into a docs PR — that is the fastest way to make a reviewer distrust every future PR you send.

How I read the function to document it



You cannot document **short_format_time** correctly by reading its name. I read the body, and the thing worth writing down was a threshold that appears nowhere else:

```
def short_format_time(t):
    if t > 60:
        return "%4.1fmin" % (t / 60.0)
    else:
        return " %5.1fs" % (t)
```

There it is: **60 seconds**. Above it, minutes. Below, seconds. And the format strings tell you the precision — one decimal place in both cases.

None of that is guessable from the outside. That is exactly what makes it worth documenting, and it is why documenting code you have not read produces something worthless.

The test for a good docstring: does it save the reader from opening the source? If they still have to look at the body to learn the threshold, you have written a label, not documentation.

The other thirty-three, concretely

To make the failure mode clear, here is the shape of a PR that did not merge. I found a function whose docstring said:

What it said	What was wrong with "improving" it
"Returns the model."	Technically accurate. Slightly terse.

I rewrote it into four lines with Args and Returns. It was not wrong. It was also not **needed** — and the maintainer now had to decide whether my four lines were better than their one, on a function nobody had complained about.

That decision has no upside for them. So it sat, and eventually the repo moved on.

An improvable docstring is a matter of taste. A missing docstring is a matter of fact. Only one of those can be resolved without a conversation, and maintainers do not have time for the conversation.

Twelve days, and what it is worth

Opened 3 July, merged 15 July by **Nanored4498** — the same maintainer who merged the Day 1 bug fix, on the same day.

That is not a coincidence, and it is the one real strategic use of a docs PR: **it puts you on a maintainer's radar in a way that costs them almost nothing**. By the time my joblib bug fix



arrived, my name was not new.

But keep the ratio honest. This works as a way into a repo you intend to keep contributing to. It does not work as a strategy on its own — thirty-five PRs proved that.

What to copy

- **Only document what is missing**, never what is merely improvable
- **Only in repos you actually use** — otherwise you are guessing
- **Match the file's existing style** exactly
- **Put the specifics in** — thresholds, formats, side effects
- **Say "no logic changes"** when true, and make sure it is

What is next

The last video: the other docstring PR that merged, in a much larger repo — and the tables in its description that made it reviewable in under a minute.

